

# SCTS: Self-Critique Tree Search for Out-of-Distribution Vulnerability Detection

Rui Zhu<sup>1,3</sup>, Nan Zhou<sup>2</sup>, Jikang Liao<sup>3</sup>, Lixin Duan<sup>1</sup>, and Guangqiang Yin<sup>2</sup>(✉)

<sup>1</sup> School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, Sichuan, 611731, China

<sup>2</sup> School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu, Sichuan, 611731, China

<sup>3</sup> Institute of Public Security, Kash Institute of Electronics and Information Industry, Kashi, Xinjiang, 844000, China

rzhu2022@std.uestc.edu.cn, zn\_986@163.com, liao.jikang@outlook.com,  
{lxduan,yingq}@uestc.edu.cn

**Abstract.** Out-of-Distribution (OOD) vulnerability detection remains a critical challenge in code security. Supervised learning models struggle to generalize to OOD vulnerability patterns. Meanwhile, Large Language Models (LLMs) show logical inconsistencies and low recall in zero-shot reasoning. These methods are limited because they simplify the iterative exploration of vulnerability analysis into a single-step prediction. To address this problem, we propose SCTS (**S**elf-**C**ritique **T**ree **S**earch) that performs iterative and self-supervised optimization at inference time. Grounded in the Monte Carlo Tree Search (MCTS) framework, SCTS enables an LLM to alternate between two roles: the Reasoner and the Critique. The Reasoner generates candidate vulnerability analysis reports. The Critique then evaluates these reports to identify logical flaws. This assessment serves as an internal reward signal that guides the search process to converge on logically consistent conclusions. We evaluated SCTS on a rigorous OOD benchmark. SCTS boosts the average recall of an 8B parameter LLM to 84.70%, outperforming a 32B parameter LLM and multiple supervised learning models. Our work demonstrates a key insight. For complex reasoning tasks like vulnerability analysis, an explicit search and self-refine mechanism is an effective path to robust OOD generalization. Our code and data are publicly available.

**Keywords:** Out-of-Distribution Generalization · Large Language Models · Self-Critique · Monte Carlo Tree Search.

## 1 Introduction

Effective detection of Out-of-Distribution (OOD) vulnerabilities is a crucial task in automated code security analysis [11,22]. Supervised learning methods follow an inductive paradigm. Their performance depends on learning known code patterns from large scale data [17]. However, the inductive bias inherent in this

approach constrains its generalization capabilities, leading to a significant performance drop [8] when encountering OOD vulnerabilities. As an alternative, zero-shot reasoning approaches based on Large Language Models (LLMs) have also failed to overcome this bottleneck [3]. Even using feed-forward reasoning strategies like Chain-of-Thought [16], these models often produce logically inconsistent outputs and suffer from low recall [23]. This issue stems from the unidirectional auto-regressive generation process. If a deviation appears in the reasoning chain, errors accumulate and propagate. The core reason is the lack of an intrinsic mechanism for backtracking [1] and self-refine [12].

Despite their different mechanisms, both technical approaches reduce the complex process of vulnerability detection to a single-step prediction task. The vulnerability detection process inherently requires iterative exploration, hypothesis generation, and logical verification. A reliable analysis must be able to evaluate multiple hypotheses, recover from errors, and converge to an optimal solution. These capabilities are cornerstone of the search paradigm [14,21]. Therefore, we reframe OOD vulnerability detection. We cast it as a problem of heuristic search within the space of reasoning paths, departing from traditional pattern matching or single-step prediction. Monte Carlo Tree Search (MCTS) offers an ideal computational framework for this problem, owing to its proven ability to efficiently navigate vast and uncertain search spaces [13]. Applying the MCTS framework to LLM reasoning presents a key challenge. It requires an effective internal reward function [20] to guide the search without external supervision.

To address the challenge, we propose SCTS (**Self-Critique Tree Search**), a self-supervised optimization algorithm that runs at inference time. The core idea of SCTS is to have the LLM play dual roles within a classic MCTS framework. First, it acts as a Reasoner to expand the reasoning path and generate new vulnerability analysis reports. Second, it serves as a Critique to evaluate these reports for logical flaws. The identified flaws are then quantified into an internal reward signal that assesses the logical rigor of a report without needing external labels. The reward signal guides the exploration and exploitation process of MCTS within the vast reasoning space. It drives the entire analysis from a logically incomplete state toward an internally consistent optimal solution.

To validate the effectiveness of SCTS, we conducted an evaluation on our OOD benchmark. This benchmark exclusively contains code with vulnerabilities from CWE categories not seen during the supervised learning models’ training phase. Experimental results show that SCTS improves the average recall of an 8B parameter LLM to 84.70%, outperforming a 32B parameter LLM and multiple supervised learning models. Our main contributions are summarized as follows:

- We proposed reframing OOD vulnerability detection as an inference time heuristic search problem, providing a new path to address the generalization limits of single-step prediction methods.
- We designed and implemented SCTS, which has a novel internal reward mechanism driven by self-critique to quantify and guide the logical quality of the LLM’s reasoning process.

- Our experiments demonstrated a key finding. A superior reasoning framework could enable a small 8B parameter model to outperform a much larger parameter model in zero-shot reasoning capability. This work provided new insights for building efficient and trustworthy AI security tools.

The remainder of this paper is organized as follows. Sect. 2 reviews related work. Sect. 3 elaborates on the SCTS framework. Sect. 4 presents our experimental setup and results. Finally, Sect. 5 concludes our study.

## 2 Related work

### 2.1 Supervised Learning-Based Vulnerability Detection

Vulnerability detection is predominantly treated as a supervised classification task. While early deep learning models often learned superficial features [2], subsequent works introduced graph-based representations (e.g., Devign [24], FGVulDet [9]) to capture code semantics via ASTs and CFGs. The advent of Pretrained Language Models (PLMs) further advanced the field. CodeBERT [4] leverages massive corpora to learn sequential and structural properties, respectively. Recent adaptations like LineVul [5], VulBERTa [6], and hybrid architectures, DetectVul [15], have optimized these models for localization, efficiency, and dependency capturing.

Despite these advancements, supervised methods inherently rely on inductive biases from known vulnerabilities, acting as pattern matchers. Consequently, they struggle with OOD scenarios. In contrast, our SCTS framework replaces static pattern matching with iterative search during inference, significantly enhancing generalization capabilities.

### 2.2 Large Language Model-Based Vulnerability Detection

Large Language Models represent a recent paradigm for vulnerability detection. Initial applications revealed that while LLMs can identify superficial patterns, they struggle with deep program logic and global context, leading to high false positive rates [7]. Although prompt engineering with structured information and Chain-of-Thought can improve accuracy [19], these methods are inherently unidirectional and lack backtracking mechanisms. To address this, a more advanced approach integrates search algorithms (e.g., Tree of Thoughts [18]) and self-refinement mechanisms (e.g., Reflexion [12]), reframing detection as an iterative exploration task. However, applying these frameworks to OOD vulnerability detection is challenging, as they require an effective reward signal to guide the search process in the absence of ground-truth labels.

our SCTS framework resolves this by generating an internal reward signal through structured self-critique. This internal reward then guides a Monte Carlo Tree Search, enabling robust label-free discovery of vulnerabilities.

### 3 Methodology

#### 3.1 Motivation

Traditional vulnerability detection is formulated as a classification problem:  $f : C \rightarrow \{0, 1\}$ , where  $C$  is the code space. However, this formulation overlooks the reasoning process essential for deriving a conclusion.

Our key insight is to reframe vulnerability detection. We transform it from a single-step prediction task into a heuristic search problem. The search performs iterative optimization within the space of reasoning paths. Consequently, a more appropriate formalization treats zero-shot vulnerability detection as a search problem. The goal is to find the optimal analysis report in the space of reasoning paths. Each report is a structured text which contains the analysis process and conclusion for a given code. We aim to design a self-supervised optimization algorithm that runs at inference time to find the optimal report  $S^*$ :

$$S^* = \arg \max_{S \in \mathcal{S}} \Phi(S, C), \quad (1)$$

where  $\mathcal{S}$  is the space of all possible reports, and  $\Phi$  is an evaluation function that measures the quality of a report  $S$  for code  $C$ .

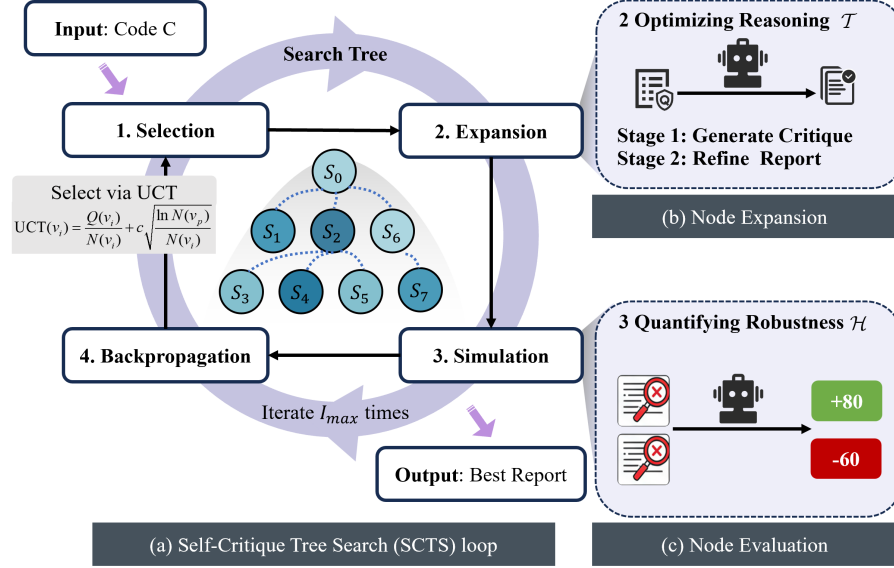
To achieve this, a self-supervised optimization algorithm should solve two key technical challenges. First, it needs a search strategy to effectively explore the vast reasoning space. Second, it requires a reward signal to evaluate reasoning quality without any external labels. To address these challenges, we propose SCTS algorithm that offers a systematic solution. It uses the classic Monte Carlo Tree Search framework as its search strategy and combines this search with an internal reward signal generated by the LLM through structured self-critique.

#### 3.2 SCTS Framework Overview

SCTS iteratively refines the analysis report of a given code snippet by searching a tree composed of reasoning states, as shown in Figure 1. In the SCTS search tree, every node  $v$  represents a reasoning state which is a complete analysis report  $S$ . The report  $S$  is a structured text that contains a judgment on the code’s vulnerability status, the type of vulnerability, and the detailed reasoning behind the conclusion. Every edge  $(u, v)$  in the tree represents a reasoning action. The action applies our structured self-critique operator  $\mathcal{T}$  to refine the parent node’s state  $S_u$  into the child node’s new state  $S_v$ .

The SCTS workflow follows the four standard MCTS steps: Selection, Expansion, Simulation, and Backpropagation. The algorithm repeats a loop until it reaches a predefined computational budget, such as a maximum number of iterations  $I_{max}$ .

**Selection.** The Selection step begins at the root node. The algorithm recursively applies the Upper Confidence Bound for Trees (UCT) policy to select child nodes. This continues until it reaches a leaf node,  $Node_{leaf}$ , that has not



**Fig. 1.** The overall architecture of SCTS.

been fully expanded. The UCT formula is:

$$UCT(v_i) = \frac{Q(v_i)}{N(v_i)} + c \sqrt{\frac{\ln N(v_p)}{N(v_i)}}, \quad (2)$$

where  $Q(v_i)$  is the node’s total reward, and  $N(v_i)$  is its visit count.  $N(v_p)$  is the parent’s visit count, and  $c$  is a constant which balances the exploitation of nodes known to yield high-quality reports against the exploration of less-traveled analysis paths..

**Expansion.** The Expansion step is the first core contribution of our work. Once a leaf node  $\text{Node}_{leaf}$  is selected, we apply a structured self-critique operator  $\mathcal{T}$ . The operator refines the node’s analysis report  $S_{leaf}$ . It produces a new improved report  $S_{new}$ . A new child node,  $\text{Node}_{new}$ , containing  $S_{new}$  is then added to the tree. The design of this operator is detailed in Sect. 3.3.

**Simulation.** The Simulation step is our second core contribution. For each new node  $\text{Node}_{new}$ , we should evaluate the report  $S_{new}$ . The evaluation generates a reward signal to guide the search. We designed a heuristic function  $\mathcal{H}$  for this purpose, as no external labels are available. The function simulates a red team audit of the report to check its logical robustness and outputs a single reward value  $R_{new}$ . We describe this function in Sect. 3.4.

**Backpropagation.** In the Backpropagation step, the reward  $R_{new}$  is propagated backward. It travels from the new node  $\text{Node}_{new}$  up to the root. The visit counts  $N(v)$  and cumulative rewards  $Q(v)$  of all nodes on this path are updated.

The loop continues until the iteration limit is reached. The algorithm’s final output is the report with the highest average reward from the entire search. We chose this strategy for a specific reason. A single evaluation using a heuristic can be noisy. However, the average reward of a node after many visits is a more robust estimate of its true quality. Selecting the report with the highest average reward is therefore a sound strategy to find the best possible solution.

### 3.3 Node Expansion: Optimizing Reasoning with Structured Self-Critique

In the expansion step of MCTS, the goal is to generate an improved report  $S_{new}$  from a parent report  $S_{parent}$ . The new report should feature more rigorous logic and deeper analysis. A naive approach of simply prompting the LLM for improvements proves unreliable. To address the challenge, we designed a structured self-critique operator  $\mathcal{T}$ . It works using a strict two-stage prompt sequence.

**Stage One.** Falsification-oriented Critique Generation. We instruct the LLM to act as a code reviewer. The role requires the model to ignore what is correct in the report. Instead, it should focus on finding logical fallacies, insufficient evidence, or overlooked edge cases. The inputs are the original code  $C$  and the report  $S_{leaf}$ . The output is a structured critique. This process compels the model to review its prior reasoning from an adversarial standpoint.

**Stage Two.** Critique-based Refinement. We give the LLM a new role of a security analyst. The model receives a prompt that includes the code  $C$ , the original report  $S_{leaf}$ , and the critique from stage one. Its task is to write a new analysis report  $S_{new}$ . The new report should fix all the flaws identified in the critique.

This structured Critique-Then-Refine process is a cornerstone of our method. It breaks an ambiguous improvement task into two well-defined subtasks. Consequently, it guides the LLM to produce a more reflective and higher quality reasoning state  $S_{new}$ .

### 3.4 Node Evaluation: Quantifying Argument Robustness with a Simulated Red Team Audit

The effectiveness of MCTS depends on the evaluation function in its Simulation step. The function should provide a reliable reward signal for new nodes to guide the tree search. Designing this function is a fundamental challenge in our zero-shot setting.

Our heuristic evaluation function  $\mathcal{H}$  is based on a core hypothesis. A report with sounder logic can better withstand targeted and adversarial questioning. In other words, the quality of a report is determined not by its superficial confidence but by its argumentative robustness. To turn this hypothesis into a computable reward, our function  $\mathcal{H}$  simulates a red team audit. The mechanism works as follows. We give the LLM a specific role that instructs the LLM to act as a red team auditor. The auditor’s only goal is to challenge every assumption in the security analysis report for the given function. After this adversarial review, the

model outputs a confidence score between  $-100$  and  $+100$ . This score measures how well the report’s conclusion holds up against the audit. We formalize this process as:

$$\text{score} = P_{\text{LLM}}(S, C, I_{\text{redteam}}), \quad (3)$$

where  $I_{\text{redteam}}$  is the prompt that instructs the red team audit. This final score serves as the reward signal  $R_{\text{new}}$  for the MCTS simulation. It provides self-supervised guidance for the SCTS search.

## 4 Experimental results

We conduct a series of experiments to address three key research questions (RQs):

RQ1. Effectiveness: Can SCTS boost the performance of LLMs on OOD tasks? Can it outperform supervised learning baselines?

RQ2. Ablation Study: What is the primary source of the performance gains from SCTS’s core components?

RQ3. Impact of Iterations: Does the performance gain of SCTS consistently increase as the number of iterations grows?

### 4.1 Experimental Setup

**Dataset and OOD Setting.** Our evaluation is conducted on the Big-Vul dataset. We simulate a strict OOD scenario for supervised models. To achieve this, we completely remove four common CWE categories from the training set: CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer), CWE-20 (Improper Input Validation), CWE-125 (Out-of-bounds Read), and CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor). Our test set is constructed to contain only vulnerable samples from these four unseen categories. This setup is designed to specifically evaluate a model’s ability to generalize to vulnerabilities outside its training distribution.

**Baseline Models.** We compare SCTS against two categories of baseline models.

**Supervised Learning Models.** These include representative GNN-based approaches, such as Devign [24] and ReGVD [10], and pretrained model approaches, such as LineVul [5]. We trained these models on the Big-Vul training set after excluding the four CWEs mentioned above.

**Vanilla LLMs.** We selected several representative large language models, including Qwen Series, Llama3.1-8B, Phi4-14B and Mistral-24. We use zero-shot prompting with these models as baselines.

**Evaluation Metrics.** Since our OOD test set contains only positive samples, Recall is our core metric. However, matching a simple label is insufficient for measuring a generative model’s true analytical power. We therefore designed a strict two-stage protocol to identify a True Positive (TP). A prediction is considered a TP only if it meets two conditions: The model’s report must explicitly set

the [Vulnerability Status] field to Vulnerable; The text in the [Reasoning] field must clearly and logically explain the vulnerability’s root cause. If the reasoning only states general knowledge and fails to analyze the specific code context, it is considered incorrect. A prediction that fails the second condition is classified as a False Negative. This protocol ensures our Recall metric reflects the model’s ability to perform grounded and interpretable vulnerability detection.

**Implementation Details.** For all SCTS experiments, the number of MCTS iterations is configured to 4 by default, unless otherwise noted. All experiments ran on a server with NVIDIA RTX 8000 GPUs. We will make our code and data publicly available to ensure reproducibility.

## 4.2 RQ1. Effectiveness

To answer RQ1, we evaluate the zero-shot recall of SCTS and baseline models on four OOD CWE categories. The results are shown in Table 1.

**Table 1.** Average recall (%) on four unseen CWEs. SCTS consistently improves LLM performance.

| Method      | Model       | CWE-119      | CWE-20       | CWE-125      | CWE-200      | Average      |
|-------------|-------------|--------------|--------------|--------------|--------------|--------------|
| Devign      | GNN         | 14.14        | 19.49        | 18.64        | 22.73        | 18.75        |
| ReGVD       | GNN         | 19.70        | 25.42        | 25.42        | 27.27        | 24.45        |
| Linevul     | CodeBert    | 63.64        | 71.19        | 61.02        | 75.00        | 67.71        |
| Vanilla LLM | Llama3.1-8B | 43.43        | 41.53        | 40.68        | 43.18        | 42.21        |
|             | Qwen3-8B    | 2.02         | 1.69         | 3.39         | 4.55         | 2.91         |
|             | Phi4-14B    | 17.68        | 12.71        | 23.73        | 29.55        | 20.92        |
|             | Qwen3-14B   | 5.08         | 3.39         | 11.86        | 6.82         | 6.79         |
|             | Mistral-24B | 61.11        | 59.32        | 79.66        | 63.63        | 65.93        |
|             | Qwen2.5-32B | 67.17        | 66.10        | <b>89.83</b> | 61.36        | 71.12        |
| SCTS(ours)  | Llama3.1-8B | 69.19        | 61.02        | 74.58        | 63.64        | 67.11        |
|             | Qwen3-8B    | <b>81.82</b> | <b>84.75</b> | 88.14        | <b>84.09</b> | <b>84.70</b> |
|             | Phi4-14B    | 69.70        | 68.64        | 74.58        | 59.09        | 68.00        |
|             | Qwen3-14B   | 80.71        | 81.36        | 86.44        | 79.55        | 82.02        |

SCTS performs exceptionally well on all OOD tasks. With Qwen3-8B, SCTS achieves a recall of over 81% on every CWE category, leading to an average recall of 84.70%, the highest among all tested methods. It outperforms the best supervised model, LineVul, by 16.99% (84.70% vs. 67.71%). Furthermore, SCTS significantly improves the performance of all base LLMs, delivering an absolute gain of +81.79% for Qwen3-8B and +24.90% for Llama3.1-8B. Notably, SCTS with Qwen3-8B also surpasses Qwen2.5-32B, a model four times its size, which scored 13.58% lower.

SCTS demonstrates a systematic advantage over the supervised learning paradigm for OOD generalization. While supervised models like LineVul perform well on in-distribution data, their OOD recall of 67.71% confirms a fundamental

limitation. In contrast, SCTS consistently improves LLM reasoning. Through its structured self-critique and search mechanism, SCTS enables an 8B LLM to outperform a 32B model.

**Answer to RQ1.** SCTS elevates LLM performance in zero-shot vulnerability detection, significantly surpassing specialized, fully supervised models. This validates the effectiveness of our proposed self-supervised, reasoning time optimization paradigm.

**Table 2.** Ablation study on Qwen3-8B. Self-critique provides the foundational performance gain, while the full MCTS search offers further systematic optimization.

| Method          | CWE-119 | CWE-20 | CWE-125 | CWE-200 | Average | Gain   |
|-----------------|---------|--------|---------|---------|---------|--------|
| Vanilla LLM     | 2.02    | 3.39   | 3.39    | 6.82    | 3.91    | -      |
| + Self-Critique | 74.75   | 76.27  | 79.66   | 72.73   | 75.85   | +71.94 |
| + Search        | 86.36   | 85.59  | 86.44   | 84.09   | 85.62   | +9.77  |

### 4.3 RQ2. Ablation Study

To isolate the contributions of its components, we conduct an incremental ablation study of SCTS using the Qwen3-8B model. The study compares three configurations:

Vanilla LLM: A zero-shot baseline without any SCTS components.

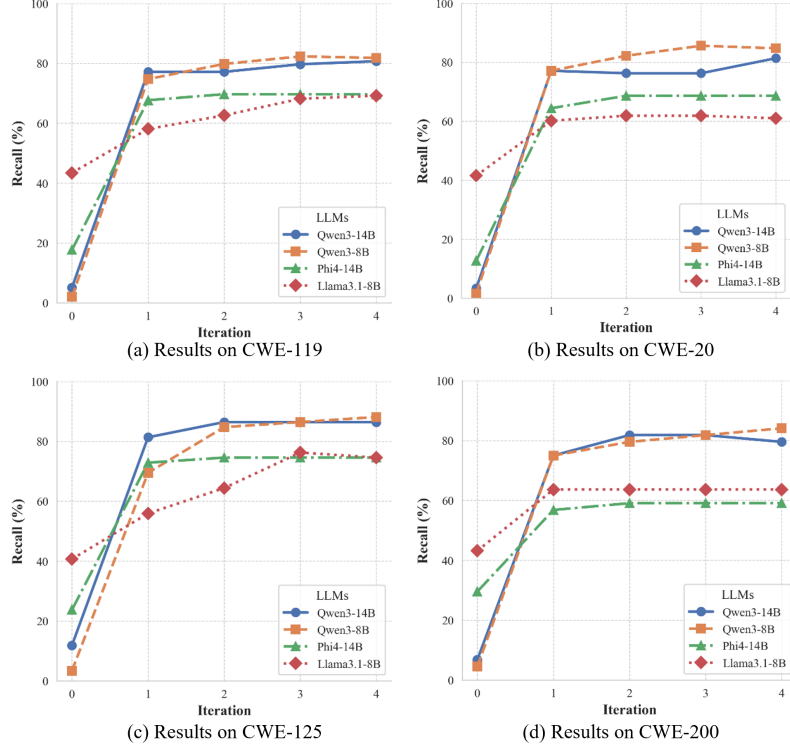
+Self-Critique: The baseline augmented with a single step of our structured self-critique operator to measure its individual impact.

+Search: The complete SCTS algorithm, including MCTS with 8 iterations.

The results in Table 2 clearly delineate the contributions of two core components. Introducing a single step of self-critique boosts the average recall from 3.91% to 75.85%. This results in a +71.94% gain. Adding the full MCTS search framework (+Search) improves the average recall further to 85.62%. This represents an additional gain of +9.77%. Although this second gain is smaller, it confirms that the systematic search from MCTS provides a distinct, valuable contribution.

These results show that our structured self-critique operator is the key driver of SCTS’s performance. It reliably transforms a low-quality analysis into a high-quality one, a critical step for enhancing the LLM’s zero-shot capability. MCTS acts as a high-level planner. It systematically explores the high-quality candidate paths produced by the critique operator. This allows the search to avoid local optima and find a globally better solution.

**Answer to RQ2.** Both core components of SCTS are integral to its success. The structured self-critique operator generates high-quality candidate solutions. The MCTS framework provides the effective exploration needed to ensure convergence to a globally optimal result.



**Fig. 2.** Average recall on the four OOD CWEs as a function of the number of MCTS iterations. SCTS consistently improves performance for all models as the computational budget increases.

#### 4.4 RQ3. Impact of Iterations

To analyze the relationship between SCTS performance and its computational budget, we evaluated recall across a range of 0 to 4 MCTS iterations for multiple LLMs.

As shown in Figure 2, the results reveal two consistent patterns. First, for all tested LLMs, the average recall increases almost monotonically as the number of MCTS iterations grows. Most of the improvement occurs within the first few iterations, after which performance begins to converge. Second, SCTS effectively boosts models with different starting performance levels. For example, Qwen3-8B starts with low performance but reaches one of the highest recall scores after 4 iterations. In parallel, Llama3.1-8B has a stronger initial performance, and SCTS also boosts its recall steadily.

**Answer to RQ3.** SCTS performance is positively correlated with its computational budget. More iterations consistently yield better results up to a saturation point, demonstrating that the performance gain is stable and predictable.

## 5 Conclusion

We introduce SCTS, a novel algorithm that tackles the critical challenge of out-of-distribution generalization in vulnerability detection. The SCTS enables an LLM to generate internal reward signals which guide a Monte Carlo Tree Search and achieve iterative optimization of reasoning without needing external labels. Our experiments demonstrate that SCTS elevates the recall of an 8B parameter LLM to 84.70%, outperforming a 32B parameter LLM and supervised learning methods. While the inference time search process introduces computational overhead, a promising direction for future work is to integrate this paradigm with efficiency enhancing techniques, such as model distillation and search pruning.

**Acknowledgments.** The authors gratefully acknowledge the support of the Natural Science Foundation of Xinjiang Uygur Autonomous Region (No.2023D01F18).

## References

- Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., et al.: Graph of thoughts: Solving elaborate problems with large language models. In: Proceedings of the AAAI conference on artificial intelligence. vol. 38, pp. 17682–17690 (2024)
- Chakraborty, S., Krishna, R., Ding, Y., Ray, B.: Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering* **48**(9), 3280–3296 (2021)
- Chen, C., Su, J., Chen, J., Wang, Y., Bi, T., Yu, J., Wang, Y., Lin, X., Chen, T., Zheng, Z.: When chatgpt meets smart contract vulnerability detection: How far are we? *ACM Transactions on Software Engineering and Methodology* **34**(4), 1–30 (2025)
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al.: Codebert: A pre-trained model for programming and natural languages. In: Findings of the Association for Computational Linguistics: EMNLP 2020. pp. 1536–1547 (2020)
- Fu, M., Tantithamthavorn, C.: Linevul: A transformer-based line-level vulnerability prediction. In: Proceedings of the 19th International Conference on Mining Software Repositories. pp. 608–620 (2022)
- Hanif, H., Maffei, S.: Vulberta: Simplified source code pre-training for vulnerability detection. In: 2022 International joint conference on neural networks (IJCNN). pp. 1–8. IEEE (2022)
- Khare, A., Dutta, S., Li, Z., Solko-Breslin, A., Alur, R., Naik, M.: Understanding the effectiveness of large language models in detecting security vulnerabilities. In: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST). pp. 103–114. IEEE (2025)
- Liu, C., Sang, Z., Duan, L., Wang, J., Ni, W., Wang, W.: Anomaly detection services for blockchain smart contracts with unknown vulnerabilities. *ACM Transactions on Software Engineering and Methodology* (2025)
- Liu, S., Ma, W., Wang, J., Xie, X., Feng, R., Liu, Y.: Enhancing code vulnerability detection via vulnerability-preserving data augmentation. In: Proceedings of the

- 25th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems. pp. 166–177 (2024)
10. Nguyen, V.A., Nguyen, D.Q., Nguyen, V., Le, T.H., Tran, T.B.K.: ReGVD: Revisiting graph neural networks for vulnerability detection. In: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings. pp. 178–182 (2022)
  11. Sheng, Z., Chen, Z., Gu, S., Huang, H., Gu, G., Huang, J.: Llms in software security: a survey of vulnerability detection techniques and insights. *ACM Computing Surveys* (2025)
  12. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* **36**, 8634–8652 (2023)
  13. Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al.: Mastering the game of go with deep neural networks and tree search. *nature* **529**(7587), 484–489 (2016)
  14. Song, X., Zhang, S., Yu, T.: Rekg-mcts: Reinforcing llm reasoning on knowledge graphs via training-free monte carlo tree search. In: Findings of the Association for Computational Linguistics: ACL 2025. pp. 9288–9306 (2025)
  15. Tran, H.C., Tran, A.D., Le, K.H.: Detectvul: A statement-level code vulnerability detection for python. *Future Generation Computer Systems* **163**, 107504 (2025)
  16. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al.: Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* **35**, 24824–24837 (2022)
  17. Yan, Y., Duong, V., Shao, H., Poshyvanyk, D.: Towards more trustworthy deep code models by enabling out-of-distribution detection. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). pp. 714–714. IEEE Computer Society (2025)
  18. Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., Narasimhan, K.: Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* **36**, 11809–11822 (2023)
  19. Zhang, C., Liu, H., Zeng, J., Yang, K., Li, Y., Li, H.: Prompt-enhanced software vulnerability detection using chatgpt. In: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings. pp. 276–277 (2024)
  20. Zhang, D., Zhoubian, S., Hu, Z., Yue, Y., Dong, Y., Tang, J.: Rest-mcts\*: Llm self-training via process reward guided tree search. *Advances in Neural Information Processing Systems* **37**, 64735–64772 (2024)
  21. Zhang, D., Huang, X., Zhou, D., Li, Y., Ouyang, W.: Accessing gpt-4 level mathematical olympiad solutions via monte carlo tree self-refine with llama-3 8b. *arXiv preprint arXiv:2406.07394* (2024)
  22. Zhou, X., Cao, S., Sun, X., Lo, D.: Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology* **34**(5), 1–31 (2025)
  23. Zhou, X., Zhang, T., Lo, D.: Large language model for vulnerability detection: Emerging results and future directions. In: Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results. pp. 47–51 (2024)
  24. Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Advances in neural information processing systems* **32** (2019)